

Fox Passport Republic API

Database Architecture • Roles & Privileges • Environments — Generated 2026-06-09

1. Roles & Privileges

Every `User` carries two independent role layers: a **SystemRole** (platform access level) and a **RoleType[]** array (marketplace business roles). A user can hold multiple business roles simultaneously.

1.1 SystemRole — platform-level access

Value	Meaning
<code>user</code>	Default for every new account. Can browse, book, and write reviews.
<code>admin</code>	Platform administrator. Full access via <code>requireAdmin</code> middleware guard.

⚠ `super_admin` is referenced in the middleware (`requireSuperAdmin`) but does **not** exist in the database enum — those guards will always reject.

1.2 RoleType[] — business / marketplace roles

Value	What it unlocks	Application required
<code>mayor</code>	Can organise events; assigned as <code>organizerId</code> on an <code>Event</code> .	<code>MayorApplication</code> — business name, TIN, BIR permit, NBI clearance, valid IDs, selfie.
<code>host</code>	Can list and own <code>Venue</code> records.	<code>HostApplication</code> — NBI clearance, portfolio file, bio, experience, location.
<code>foxerAsset</code>	Can list and own <code>Asset</code> records (equipment, furniture, etc.).	<code>FoxerAssetApplication</code> — business name, asset types[], TIN, BIR permit, NBI, valid IDs.
<code>foxerService</code>	Can list and own <code>Service</code> records (catering, entertainment, etc.).	<code>FoxerServiceApplication</code> — service types[], portfolio URLs, years of experience, NBI clearance number.
<code>investor</code>	Investor tier access on the platform.	<code>InvestorApplication</code> — investment range, interests[], proof of funds file.

1.3 Role Request Lifecycle

Users submit a `RoleRequest` which starts at `pending` and transitions to `approved` or `rejected`. On approval, the role is appended to `User.roleType[]`. Each request carries exactly one

application detail record (one-to-one with the appropriate application model).

1.4 Middleware Guards

Guard	Who passes
<code>authenticate</code>	Any user with a valid, non-expired JWT Bearer token.
<code>optionalAuth</code>	Everyone — attaches user context if a token is present, but doesn't block unauthenticated requests.
<code>requireAdmin</code>	<code>systemRole === admin</code> (or <code>super_admin</code> , though that value can't exist).
<code>requireHost</code>	<code>roleType</code> includes <code>host</code> , OR <code>systemRole === admin</code> .
<code>requireRole([...])</code>	Any match in the user's <code>systemRole</code> OR <code>roleType</code> array against the provided list.
<code>requireOwnerOrAdmin</code>	User is the resource owner OR has admin <code>systemRole</code> . Used on profile and asset edit endpoints.

2. Database Relations

2.1 One-to-One

Parent	Child	Notes
User	Passport	Gamification record — stamps, mileage, level, achievements JSON. Cascade-deleted with user.
RoleRequest	MayorApplication	Exactly one application detail per request. Cascade-deleted with request.
RoleRequest	FoxerAssetApplication	Same pattern as above.
RoleRequest	FoxerServiceApplication	Same pattern as above.
RoleRequest	InvestorApplication	Same pattern as above.
RoleRequest	HostApplication	Same pattern as above.

2.2 One-to-Many

Parent	Children	Notes
User	Asset[]	A <code>foxerAsset</code> user owns many assets. Cascade-deleted with user.
User	Service[]	A <code>foxerService</code> user owns many services. Cascade-deleted with user.
User	Venue[]	A <code>host</code> user owns many venues. Cascade-deleted with user.
User	EventTemplate[]	Reusable event blueprints owned per user.
User	Booking[]	A user can have many event bookings.
User	Review[]	A user can write many reviews (polymorphic target).
User	Favorite[]	A user can save many favorites (polymorphic target).
User (Applicant)	RoleRequest[]	A user can submit many role upgrade requests over time.
User (Reviewer)	RoleRequest[]	An admin can review many role requests.
User (Client)	Event[]	Named relation <code>EventClient</code> — the user who pays for the event.

Parent	Children	Notes
User (Organizer)	Event[]	Named relation <code>EventOrganizer</code> — the mayor user who runs the event.
User	BookingAttendee[]	User appears as an attendee on bookings.
User (Inviter)	BookingAttendee[]	User who invited other attendees to a booking.
User	EventAssetTransaction[]	As asset provider — tracks assets the user supplied to events.
User	EventServiceTransaction[]	As service provider — tracks services the user provided.
User	EventVenueTransaction[]	As venue provider — tracks venues the user rented out.
EventTemplate	Event[]	A template can spawn many actual events.
Event	Booking[]	Multiple attendees/groups can book the same event.
Event	EventAssetTransaction[]	All asset usage records for an event. Cascade-deleted.
Event	EventServiceTransaction[]	All service usage records for an event. Cascade-deleted.
Event	EventVenueTransaction[]	All venue usage records for an event. Cascade-deleted.
Booking	BookingAttendee[]	The guest list for a booking. Cascade-deleted.
Booking	Payment[]	Multiple payment records per booking (deposit + full). Cascade-deleted.
Asset	AssetBooking[]	Direct standalone bookings of an asset (outside of events).
Service	ServiceBooking[]	Direct standalone bookings of a service (outside of events).
Asset	File[]	Gallery images for the asset listing.
Service	File[]	Gallery images for the service listing.
Venue	File[]	Gallery images for the venue listing.
EventTemplate	File[]	Images attached to the template.

2.3 Many-to-Many (explicit junction tables with business data)

These are not simple pivot tables — each junction carries pricing and matching metadata:

Junction Table	Left	Right	Extra fields
EventTemplateAsset	EventTemplate	Asset	agreedPrice , quantity , isOptional , matched , matchConstraint , matchedAt , date
EventTemplateService	EventTemplate	Service	agreedPrice , isOptional , matched , matchConstraint , matchedAt , date
EventTemplateVenue	EventTemplate	Venue	agreedPrice , isOptional , matched , matchConstraint , matchedAt , date

The `matched` flag and `matchConstraint` enum power the matching engine (`match.service.ts`), which auto-assigns real providers to template slots. Constraints:

MatchConstraint	Meaning
<code>SAME_STATE</code>	Provider must be in the same state as the event target.
<code>BROADER</code>	Match across a wider geographic area.
<code>MANUAL_OVERRIDE</code>	Admin manually assigned a provider.
<code>NONE</code>	No geographic constraint (default).

2.4 Polymorphic Relations

Two models use a **string-discriminated polymorphic pattern** — they store an `entityId` + `entityType` string instead of typed foreign keys. No FK enforcement at the DB level.

Model	Pattern	Possible entityType values
Review	<code>userId</code> + <code>entityId</code> + <code>entityType</code>	venue, asset, service, event, etc. Unique constraint on (userId, entityId, entityType).
Favorite	<code>userId</code> + <code>entityId</code> + <code>entityType</code>	Same — one table covers all saveable entity types.

2.5 Self-referencing on User

Relation	How
Reviewer → Applicant	<code>RoleRequest.reviewer</code> and <code>RoleRequest.user</code> both point to <code>User</code> with named relations.

Relation	How
Inviter → Attendee	<code>BookingAttendee.invitedBy</code> points back to <code>User</code> . The attendee may or may not have a <code>userId</code> (supports guest invites by name/email).
Client vs Organizer on Event	One <code>Event</code> has two <code>User</code> FK references: <code>clientId</code> (who pays) and <code>organizerId</code> (the mayor who plans it).

2.6 File — Shared Upload Table

`File` is a central file registry. It carries nullable FKs to `Venue`, `Asset`, `Service`, `EventTemplate`, and `User`. Application detail models (Mayor, Host, FoxerAsset, etc.) reference `File` by named relations for each document type (NBI, BIR permit, TIN, selfie, valid ID, portfolio). A single `File` row can be referenced by multiple application records across different roles, which is why those back-relations are arrays.

3. Transaction Flow (Events)

When an event is executed, three transaction tables record the financial/operational commitments:

Table	Links	Key fields
<code>EventAssetTransaction</code>	Event + Asset + Provider (User) + optional Booking	<code>agreedPrice</code> , <code>quantity</code> , <code>status</code> (pending/approved/rejected/cancelled), <code>lockedAt</code> , <code>releasedAt</code> , <code>included</code>
<code>EventServiceTransaction</code>	Event + Service + Provider (User) + optional Booking	Same status/price/lock pattern as above.
<code>EventVenueTransaction</code>	Event + Venue + Provider (User) + optional Booking	Same status/price/lock pattern as above.

A `Booking` can be optionally linked to these transactions (via `bookingId`), letting you see which specific booking triggered which transaction. `Payments` belong to `Booking` with two types: `deposit` and `full` .

4. Environments

Setting	Development (<code>.env</code>)	Docker (<code>.env.docker</code>)
API Port	3002	3002
NODE_ENV	development	production
DB Host	<code>localhost:5432</code>	<code>postgres:5432</code> (container hostname)
DB Name	<code>foxpassportrepublic</code>	<code>fox_passport_db</code>
DB User	<code>admin</code>	<code>postgres</code>
Redis Host	<code>localhost</code>	<code>redis</code> (container hostname)
Redis Port	<code>6379</code> (host-mapped from Docker's <code>6378:6379</code>)	<code>6379</code> (internal)
Storage	AWS S3 + CloudFront	Supabase Storage (placeholder)

Setting	Development (<code>.env</code>)	Docker (<code>.env.docker</code>)
pgAdmin	Not used	<code>localhost:5050</code> (container)

4.1 Docker Service Map

Service	Image	Host Port	Container Port	Name
PostgreSQL	postgres:15	5432	5432	local_postgres
Redis	redis:7	6379	6379	local_redis
pgAdmin 4	dpage/pgadmin4	5050	80	local_pgadmin